

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ULAŞIMDA YAPAY ZEKA YARIŞMASI

KRİTİK TASARIM RAPORU

TAKIM ADI

T-KOD SOFTWARE

BAŞVURU ID

#315673

İçindekiler Tablosu

1.RAPOR ÖZETİ.....	3
2.TAKIM ŞEMASI	4
3. PROJE MEVCUT DURUM DEĞERLENDİRMESİ	4
4. ALGORİTMALAR VE SİSTEM MİMARİSİ.....	5
4.1. Veri Setleri.....	5
4.1.1. Generative Adversarial Networks (GAN)	5
4.2. Algoritmalar	6
4.2.1. Evrimsel Sinir Ağları (CNN).....	7
5.SİSTEM MİMARİSİ.....	8
6. ÖZGÜNLÜK.....	8
6.1. SAHI Algoritması.....	8
6.2. Deep Sort Algoritması	9
6.2.1. Deep Sort Algoritması Nasıl Çalışır?	9
6.2.1.1 Nesne Tespiti.....	9
6.2.1.2. Modelleme	9
6.2.1.3. Nesne İlişkilendirilmesi.....	9
6.3. C++ Yazılım Dili.....	10
6.3.1. C++ Nedir ?	10
7. SONUÇLAR VE İNCELEME.....	11
7.1. Veri Toplama.....	11
7.2. Veri Etiketleme.....	11
7.3. Uygun Yapay Zeka Modelinin Belirlenmesi.....	11
8. KAYNAKÇA.....	12

ŞEKİL LİSTESİ

Şekil 1: Takım Şeması.....	4
Şekil 2: GAN Çalışma Prensibi.....	5
Şekil 3: YOLO Algoritma Çeşitleri ve Farkları	6
Şekil 4: Veri Setinden Alınan Örnek	6
Şekil 5: Teknofest'in Paylaştığı Videodan Alınan Görsel	7
Şekil 6: CNN Çalışma Prensibi.....	7
Şekil 7: Sahi Algoritması Kullanılmayan Görsel.....	8
Şekil 8: Sahi Algoritması Kullanılan Görsel.....	8
Şekil 9: Nesnelerin İlişkilendirilmesi Örnek.....	9
Şekil 10: Örnek IoU Numaralandırması.....	10
Şekil 11: Yazılım Dilleri.....	10
Şekil 12: Örnek Drone Görüntüsü.....	11
Şekil 13: Make.Sense Arayüz.....	11

Şekil 14: Tespit Edilen Araçlar.....	11
--------------------------------------	----

TABLO LİSTESİ

Tablo 1: Sistem Mimarisi	8
--------------------------------	---

KISALTMALAR

CNN -- Convolutional Neural Network

YOLO -- You Only Look One

CARPK -- Car Parking Lot Dataset

GAN -- Generative Adversarial Networks

SAHI -- Slicing Aided Hyper Inference

IoU -- Intersection Over Union

JSON -- JavaScript Object Notation

1. RAPOR ÖZETİ

Bu raporda Yapay zeka ile nesne tanıma uygulamalarının ulaşım için nasıl kullanılabileceği ve bu uygulamalar için geliştirilen modelin ar-ge çalışması yapılırken yaşanan sorunlar, tecrübeler ve çözüm önerileri yer almaktadır. Nesne tanıma için geliştirilen bir yapay zeka modeli için kullanılacak olan veri setinin seçimi için önemli noktalardan biri problemin kendisine yatmaktadır. Kullanılacak olan nesne tanıma algoritmasının öncelikle karşılaşılabilecek durumlar iyi belirlenmeli ve bu durumlara uygun veriler toplanmalıdır. Yeterli veri yok ise mevcut verileri kullanarak yine bir yapay zeka örneği olan GAN algoritması kullanılıp makine ile daha çok veri üretilebilir. Ne kadar çok veri toplanırsa okadar yüksek tutarlılık sağlanır. CNN algoritmasının çalışma prensibi ve bu algoritmanın bazı durumlarda tek başına yeterli olmaması ile birlikte kullanılan yardımcı algoritmalar kullanılması gerekebilir örneğin SAHI algoritması kullanarak, kamerada çok küçük bir alan kaplayan nesneler bile tespit edilebilir. Bunun yanında Deep Sort algoritması hareketli olan nesnelerin takip edilebilmesini sağlar. Ulaşım gibi sürekli hareketli bir ortamda özellikle çok iş görmektedir. Yapay zeka modeli tutarlı olduğu kadar hızlıda çalışması gerekir ve kısıtlı donanıma sahip bir alanda iyi performans çok önemlidir. Bu yüzden kullanılan algoritmalar oldukça iyi optimize edilmeli C++ gibi sürekli gelişen ve üstüne koyulan bir yazılım dili gibi çözümler tercih edilmelidir.

2. TAKIM ŞEMASI



Şekil 1: Takım Şeması

3. PROJE MEVCUT DURUM DEĞERLENDİRMESİ

Yapay zeka ile nesne tespiti projesi ön tasarım raporun da belirtilen takvime uygun olarak gelişmeye devam etmektedir. Öncelikle ön tasarım raporunda belirtmiş olduğumuz algoritmalar (CNN ve YOLOv5) arasında yaptığımız testler sonucunda yarışmaya YOLOX kullanarak devam etme kararı alınmıştır. YOLOX algoritması diğer mimarilere göre oldukça hızlı çalışması ve sonuç üretmesi bu fikirde devam edilmesine sebep olmuştur. Ayrıca veri seti toplama çalışmaları halen sürmektedir. Teknik şartnamede paylaşılan iniş alanı görüntüleri kullanılarak yeni veriler oluşturulmuştur. Ayrıca modelin tespit etmekte zorlandığı görüntüler konusunda detaylı çalışılmaktadır, araçların etiketleme çalışmaları devam etmektedir.

4. ALGORİTMALAR VE SİSTEM MİMARİSİ

4.1. Veri Setleri

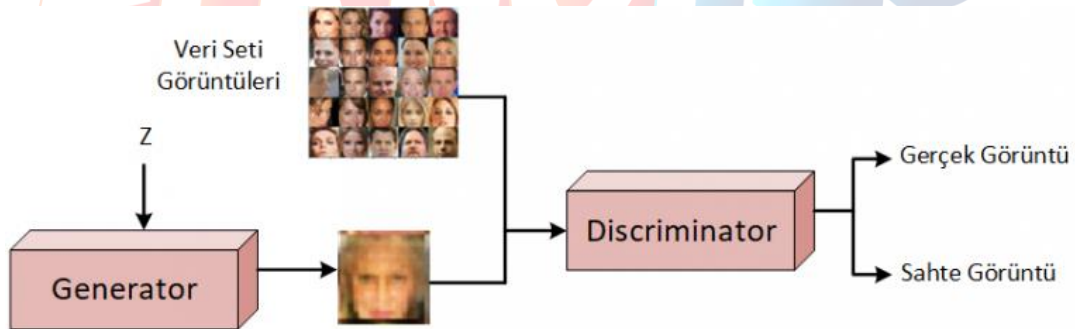
Veri setleri internetteki birçok kaynaktan toplanmıştır. Toplanan veri setlerinden biri olan Otopark Veri Kümesi (CARPK) kullanılmıştır. Veri kümesindeki fotoğraflar etiketlenip model eğitilmiştir fakat taşıt görüntülerinin çok fazla dik ve yüksek bir mesafeden çekilmiş olmasından dolayı, eğitilen model Teknofest'in göndermiş olduğu örnek videoda istenen performansı verememiştir. Bu yüzden Örnek videodaki görüntülere daha yakın olan VisDrone veri seti ağırlıklı kullanılarak yeniden model eğitilecektir. Aynı zamanda şartnamede belirtilen yarışmada verilecek görüntülerin sahip olduğu hava olayları (karlı, yağmurlu, güneşli sisli), 60 ve 90 derece açı kriterleri, kısmen görünen taşıt ve insan kareleri, aynı ortamda bulunan birden fazla çeşitli taşıt ve insan gölgeleri özelliklerine de daha çok uymaktadır. Model eğitimine destek için Generative Adversarial Networks (GAN) algoritmasında kullanılarak eldeki veri setini hem çoğaltmış hem de örnek videodakine benzer daha çok veri elde edebilir böylece daha tutarlı bir model eğitebiliriz.

4.1.1. Generative Adversarial Networks (GAN)

Çekişmeli üretici ağ (generative adversarial network ; GAN), Ian Goodfellow ve meslektaşları tarafından 2014 yılında tasarlanan bir makine öğrenimi framework sınıfıdır. Bir oyunda iki sinir ağı birbirleriyle yarışmaktadır (Bir ajanın kazancının diğer bir ajanın kaybının olduğu sıfır toplamı bir oyun şeklinde bir oyundur.).

Bir eğitim seti verildiğinde, bu teknik ile eğitim seti aynı istatistiklerle yeni veriler oluşturmayı öğrenmektedir. Örneğin, fotoğraflar üzerinden eğitilmiş bir GAN, insan gözlemi için bile birçok gerçekçi özelliğe sahip ve yüzeysel olarak özgün görünen yeni fotoğraflar oluşturabilmektedir. Başlangıçta denetimsiz öğrenme için üretken bir model olarak önerilmiş olsa da, çekişmeli üretici ağların yarı denetimli öğrenme, tam denetimli öğrenme ve pekiştirmeli öğrenme için de yararlı olduğu kanıtlanmıştır.

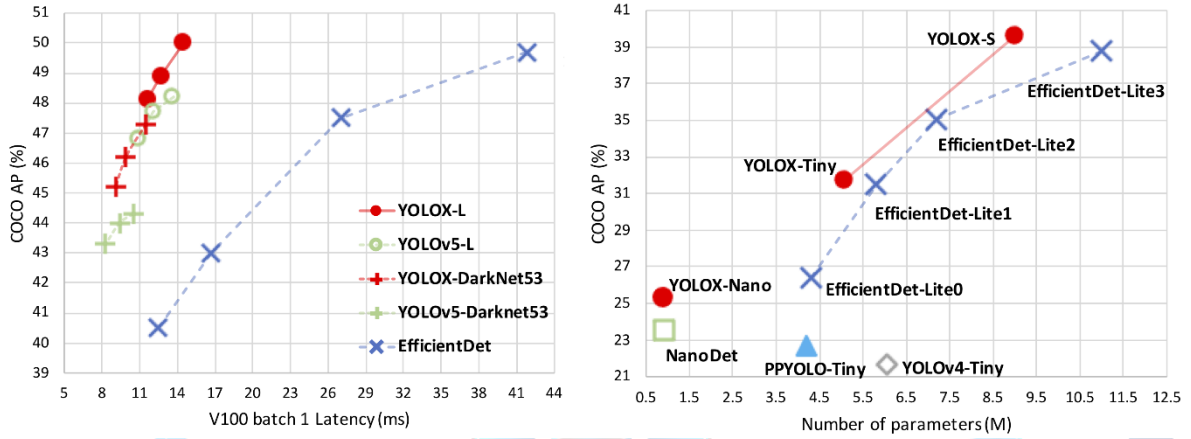
Bir GAN'in temel fikri, kendisi de dinamik olarak güncellenen ayrıştırıcı aracılığıyla "dolaylı" eğitime dayanmaktadır. Bu temelde, üreticinin belirli bir görüntüye olan mesafeyi en aza indirecek şekilde eğitilmediği, bunun yerine ayrıştırıcıyı kandırmak için eğitildiği anlamına gelmektedir. Bu, modelin denetimsiz bir şekilde öğrenmesini sağlamaktadır.



Şekil 2: GAN Çalışma Prensibi

4.2. Algoritmalar

Nesne tespiti yapmak için Python yazılım dili ile birlikte CNN (Convolutional Neural Networks) algoritmasını kullanan Yolov5 Kullanılması planlanmıştır. Fotoğraflardaki, videolardaki ve gerçek zamanlı görüntülerdeki nesneleri tespit etmeye odaklanan bilgisayarlı görü ve görüntü işleme algoritması olan YOLO konvolüsyonel sinir ağıları kullanarak nesne tespiti yapan bir algoritmadır. Açılımı "You Only Look Once" demektir. YOLO Kullanılmasının temel sebeplerinden biri kullanımının basit olması hızlı ve tutarlı çalışması aynı zamanda şu anda piyasada bulunan son teknolojilerden biri olmasıdır. YOLO algoritmasının diğer algoritmalarından daha hızlı olmasının sebebi resmin tamamını tek seferde nöral bir ağdan geçiriyor olmasıdır. Ancak yapılan testler sonucunda Yolov5, eğittiğimiz veri setindeki taşıtlarda yüksek performansla tespit yapılmasına rağmen Teknofest'in örnek olarak paylaştığı videoda istediğimiz sonucu verememiştir. Bu yüzden Yolov5 Modelinden daha gelişmiş olan YoloX Modeli kullanılacaktır. Aşağıdaki Grafiklerde de gösterildiği gibi YoloX hem performans hem de tutarlılık olarak Yolov5 ten daha gelişmiş bir düzeydedir.



Şekil 3: YOLO Algoritma Çeşitleri ve Farkları

Aşağıdaki görüntülerde yolov5 ile eğittiğimiz modelin veri seti üzerindeki ve Teknofest'in paylaştığı videodan alınan bir fotoğraftaki performansı görülmektedir.



Şekil 4: Veri Setinden Alınan Örnek



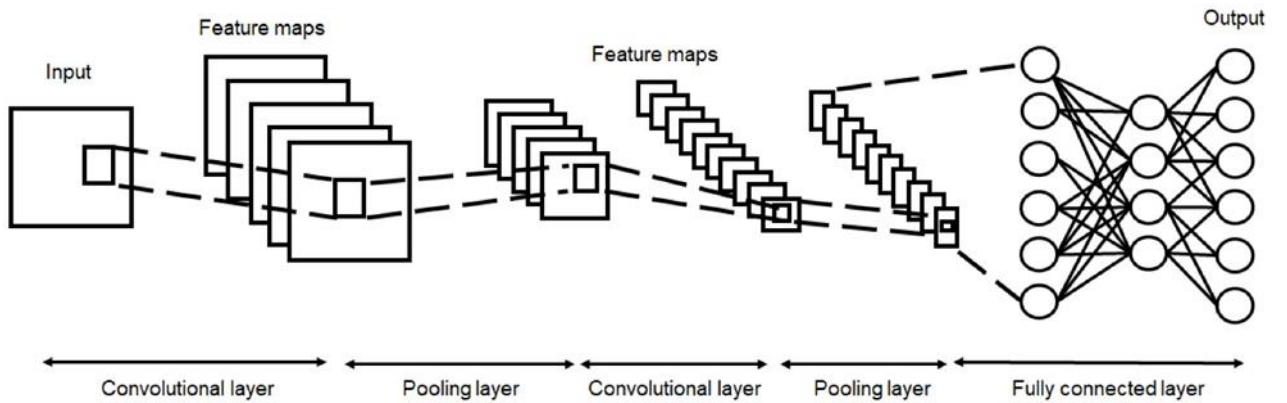
Şekil 5: Teknofest'in Paylaştığı Videodan Alınan Görsel

Fark edildiği üzere veri setinden alınan görselde çok iyi performans vermesine rağmen teknofest'in paylaştığı videodan alınan görselde olduğu üzere hatalar meydana gelmesinden dolayı yolov5 ten vazgeçip daha tutarlı ve hızlı tespit yapabilen yolox modeli tercih edilmiştir.

4.2.1. Evrişimsel Sinir Ağları (CNN)

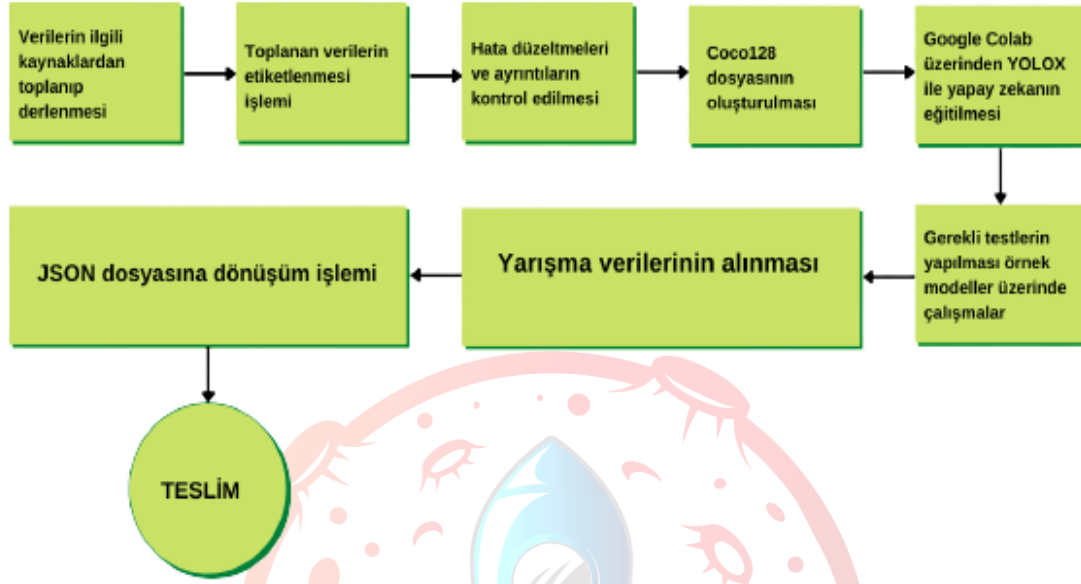
Evrişimsel Sinir Ağları (CNN), girdi verisi olarak görüntüleri kullanan bir derin öğrenme algoritmasıdır. Görüntülerin özelliklerini (features) kullanarak görüntüleri birbirinden ayırt etmektedir diğer bir deyişle sınıflandırma işlemini gerçekleştirmektedir. CNN de kullanılacak veriler için gerekli olan ön işleme, diğer sınıflandırma algoritmalarına kıyasla çok daha düşüktür. Bunun sebebi ise CNN' in diğer sınıflandırma algoritmalarından farklı olarak filtreleri öğrenme yeteneğine sahiptir. CNN mimarisi temel olarak üç katmandan oluşur. Evrişimli Katman (Convolutional Layer), Havuzlama Katmanı (Pooling Layer) ve Tam Bağlantılı Katman (Fully Connected Layer). Bu katmanlardan geçen görsel, farklı işlemlerden geçirilerek derin öğrenme modeline girecek hale gelir.

- **Convolutional Layer** — Özellikleri saptamak için kullanılır.
- **Non-Linearity Layer** — Sisteme doğrusal olmayanlığın (non-linearity) tanıtılması
- **Pooling (Downsampling) Layer** — Ağırlık sayısını azaltır ve uygunluğu kontrol eder.
- **Flattening Layer** — Klasik Sinir Ağı için verileri hazırlar.
- **Fully-Connected Layer** — Sınıflamada kullanılan Standart Sinir Ağı



Şekil 6: CNN Çalışma Prensibi

5. SİSTEM MİMARİSİ



Tablo 1: Sistem Mimarisi

6. ÖZGÜNLÜK

6.1. SAHI Algoritması

Küçük nesnelerin ve uzaktaki nesnelerin algılanması, yapay zeka modelleri için büyük bir zorluktur. Bu tür nesneler görüntüde az sayıda pikselle temsil edilir ve yeterli ayrıntıya sahip değildir, bu da geleneksel dedektörler kullanılarak tespit edilmelerini zorlaştırır. Bu nedenle, (SAHI) adı verilen ve küçük nesne tespiti için geliştirilmiş algoritma kullanılmıştır.



Şekil 7: Sahi Algoritması Kullanılmayan Görsel



Şekil 8: Sahi Algoritması Kullanılan Görsel

6.2. Deep Sort Algoritması

Deep Sort (Derin sıralama) algoritması tespit edilen nesnenin bir sonraki frame de bulunduğu noktayı baz alarak nesnenin gitti yönü tespit edip takibini gerçekleştiren bu sayede daha yüksek tutarlılıkta yapay zekanın nesne tespiti yapmasını sağlayan bir algoritmadır. Eğittiğimiz modele yardımcı olarak nesne tespit yazılımına Deep Sort algoritması kullanılmıştır.

6.2.1. Deep Sort Algoritması Nasıl Çalışır?

6.2.1.1. Nesne Tespiti

Nesne takibinin yapılabilmesi için ilk önce nesne tespiti yapılır. Tespit edilen nesne geometrik bir şekil içine alınır ve daha önce kullanılmamış bir numara atanır. Nesnenin takibi bu numara sayesinde yapılır. İlk defa tespit edilen nesnenin hızı da sıfır olarak belirlenir.

6.2.1.2. Modelleme

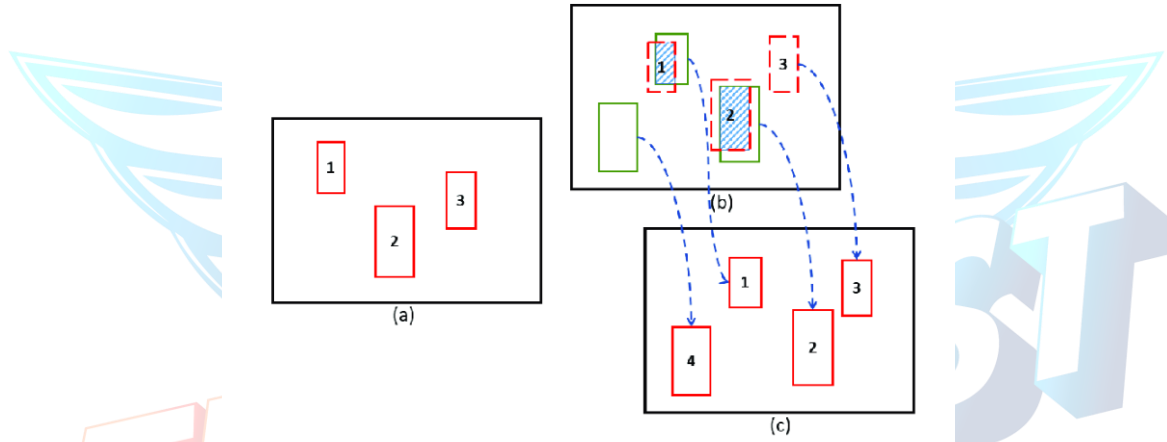
Tespit edilen nesnenin modellemesi ise aşağıdaki gibidir:

$$x = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]$$

Burada u ve v , hedefin merkezinin yatay ve dikey piksel konumunu temsil ederken, s ve r ölçeği sırasıyla hedefin sınırlayıcı kutusunun ölçeğini (alan) ve en boy oranını temsil eder.

En ve boy oranı sabit kalmaktadır. Nesnenin sonraki imgede nerede bulunacağı Kalman Filter yöntemi ile belirlenir ve koordinatlar da ona göre güncellenir.

6.2.1.3. Nesne İlişkilendirilmesi



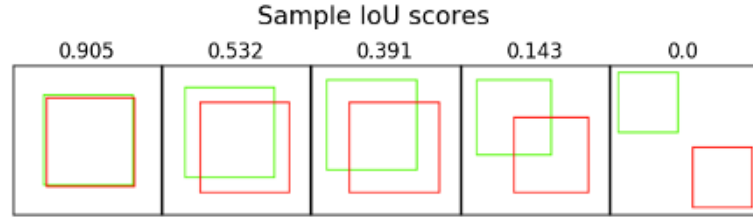
Şekil 9: Nesnelerin İlişkilendirilmesi Örnek

Bir önceki imgede bulunan nesne ile şu anki imgede bulunan nesnenin aynı nesne olup olmadığını belirlenmesine yarar. Hungarian algoritmasını kullanarak iki nesne arasında ilişki kurulabilir. İki nesneyi kıyaslarken elde edilen skorlara göre nesne ilişkilendirmesi yapılır.

Skorların hesaplanması için farklı yöntemler vardır:

- **IoU (Intersection Over Union)**
- **Shape Score**
- **Convolution Cost**

SORT algoritmasında ise IoU yöntemi kullanılır. Daha önce tespit edilen ve sonraki yeri tahmin edilen nesnelerin kesişimine bakılarak nesne ilişkilendirilmesi yapılır. Aynı olan nesne için kesişim maksimize edilmeye çalışılır.

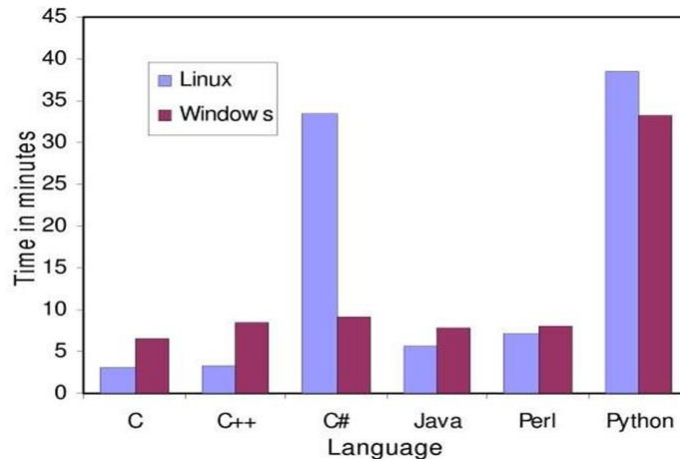


Şekil 10: Örnek IoU Numaralandırması

Takibi yapılan nesne sonraki imgede rastlanmaz, yanlış tespit edilir ya da herhangi bir nedenden ötürü görünmez ise nesnenin imgeden çıktığı kabul edilir. Nesne tekrar görünmeye başladığında ise yeni numara atanır.

6.3. C++ Yazılım Dili

Model eğitilirken kullanılacak yazılım dilinin Python olmasının sebebi kullanışı kolay ve prototipleme için uygun olmasıdır. Bu özellikler yanında bir dezavantajı getirmektedir o ise yavaş çalışmasıdır. Yarışmada yüksek doğrulukla aynı zamanda hızlı tespit yapılması gerektiğinden model eğitilip ağırlık değerleri elde edildikten sonra resimdeki istenen nesnenin tespiti ve koordinatlarının çıkarılıp JSON dosyası olarak kaydedilme işlemi C++ yazılım dili ve OpenCV kütüphanesinin yardımı ile Linux işletim sisteminde yapılacaktır. Bunun sebebi, C++ yazılım dilinin, Python yazılım diline göre 8 kat daha hızlı çalışmasıdır. Bu avantajla birlikte, nesne tanıma yazılımına daha çok yardımcı algoritma eklenecek ve performans düşmesi probleminin önüne geçilecektir. Aşağıdaki tabloda C++ ile Python arasındaki çalışma hızı farkı açıkça görülmektedir.



Şekil 11: Yazılım Dilleri

6.3.1. C++ Nedir ?

C++, Bell Laboratuvarlarından Bjarne Stroustrup tarafından 1979 yılından itibaren geliştirilmeye başlanmış, C'yi kapsayan ve çok paradigmatlı, yaygın olarak kullanılan, genel amaçlı bir programlama dilidir. İlk olarak C With Classes (Sınıflarla C) olarak adlandırılmış, 1983 yılında ismi C++ olarak değiştirilmiştir.

Genel olarak her C programı aynı zamanda bir C++ programıdır, ancak her C++ programı bir C programı değildir. Bu durumun bazı istisnaları mevcuttur. C++'ı C'den ayıran özellikler C++'ın nesne paradigması kullanılarak programlamaya olanak tanıyan özelliklerdir. Sınıflar sayesinde yeni veri türleri yaratılabilir veya varolan türlerden yenileri türetilir. Ayrıca çokbüçümlilik sayesinde bir sınıf tanımlı yazılmış kod, o sınıf türünden türetilmiş yeni sınıflarla da çalışabilir.

7. SONUÇLAR VE İNCELEME

7.1. Veri Toplama

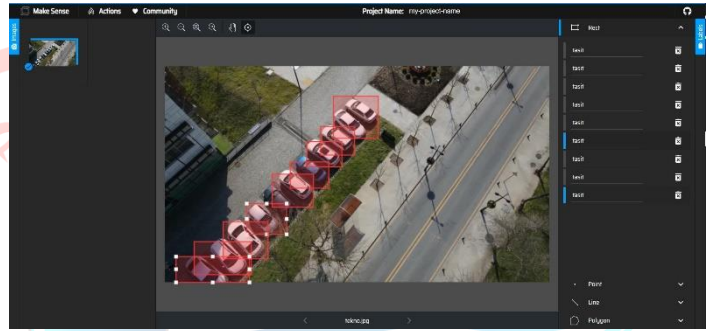


Şekil 12: Örnek Drone Görüntüsü

Veri toplama sürecinde drone kiralanıp Teknofest'in paylaştığı videodaki görüntülere yakın görseller kaydedilmek istenmiştir. Ancak Şehir alanında izinsiz drone uçuşunun yasak ve tehlikeli olmasından dolayı. Bulunduğumuz bölgedeki kaymakamlıktan izin alınması gerekliliği oluşmuştur bu izin sürecinin sebep olacağı zaman kaybından dolayı bu yöntemden vazgeçilmiş ve hazır veri setleri ile çalışılmaya devam edilmiştir.

7.2. Veri Etiketleme

Veri etiketleme sürecinde kullanmış olduğumuz veri setlerinin çok büyük olması ve her görselde yaklaşık 40 tane etiketlenmesi gereken nesnenin olması sürecin çok yavaş ilerlemesine aynı zamanda yapay zeka modeli denemelerinin gecikmesine sebep olmuştur. Bu yüzden birden fazla kişiye görev dağılımı yapılarak süreç hızlandırılmıştır. Ancak veri etiketleme sürecinde dikkat edilmeyen bazı hususlar, nesnenin tamamının etiket alanına girmemesi, etiketlenen nesnenin dışında daha büyük bir alanın işaretlenmesi ya da tek tek nesneleri işaretlemek yerine birbirine yakın duran nesneleri tek bir nesne gibi etiketleme hataları yapıldığı gözlemlenmiştir. Bu sonuçlardan etiket yapan kullanıcıların detayları iyi bilmesi ve ona göre görevi yerine getirmesi gerektiği anlaşılmıştır.



Şekil 13: Make.Sense Arayüz

7.3. Uygun Yapay Zeka Modelinin Belirlenmesi



Şekil 14: Tespit Edilen Araçlar

Uygun yapay zeka modeli belirleme sürecinde yarışma süresi göz önüne alınarak hızlı bir model olan YOLOV5 ile test çalışmalarına başlanmıştır. Ancak yapılan testler sonucunda veri seti içerisindeki örneklerde çok iyi sonuç alınmasına rağmen Teknofest'in vermiş olduğu videoda tutarlılığın çok düşük olduğu görülmüştür. Bu yüzden daha yavaş çalışan ve tutarlılığı yüksek bir model denenmiştir. Ancak yavaş olan modelin yarışma süresini aşması ve yine istenen sonucu verememesinden dolayı yapılan araştırmalar sonucunda YOLOV5'in daha üst modeli olan YOLOX'in kullanılmasına ek olarak SAHI ve Deep Sort gibi yardımcı algoritmalara daha çok ağırlık

verilmiştir.

8. KAYNAKÇA

- [1] <https://www.makesense.ai/>
- [2] <https://deryacakmak.medium.com/sort-ve-deep-sort-algoritmalar%C4%B1-7f50c246eac7#:~:text=CNN%20yap%C4%B1s%C4%B1n%C4%B1n%20son%20katman%C4%B1nda%20nesnelerin,kullan%C4%B1larak%20iki%20nesnenin%20ili%C5%9Fkilendirilmesi%20yap%C4%B1%C4%B1r>
- [3] <https://github.com/kadarnar/yolov5-pytorch-sahi>
- [4] <https://github.com/Megvii-BaseDetection/YOLOX>
- [5] <https://medium.com/@tuncerergin/convolutional-neural-network-convnet-yadacnn-nedir-nasil-calisir-97a0f5d34cad>
- [6] <https://medium.com/deep-learning-from-deepest/deri%CC%87n%C3%B6%C4%9Frenme-ve-evri%CC%87%C5%9Fi%CC%87mselsi%CC%87ni%CC%87r-a%C4%9Flari-446dc8a8d2f>
- [7] <https://www.datascienceearth.com/veri-bilimi-icin-kaggle/>
- [8] <https://medium.com/deep-learning-from-deepest/deri%CC%87n%C3%B6%C4%9Frenme-ve-evri%CC%87%C5%9Fi%CC%87mselsi%CC%87ni%CC%87r-a%C4%9Flari-446dc8a8d2f>
- [9] <https://www.smartera.com.tr/gercek-zamanli-nesne-takibireal-time-objectdetection-w-yolo-python/>
- [10] <https://github.com/obss/sahi>
- [11] <https://stanford.edu/~shervine/l/tr/teaching/cs-230/cheatsheet-convolutionalneural-networks>
- [12] <https://ayyucekizrak.medium.com/deri%CC%87ne-daha-deri%CC%87neevri%C5%9Fimli-sinir-a%C4%9Flar%C4%B1-2813a2c8b2a9>
- [13] <https://ayyucekizrak.medium.com/veri-odakl%C4%B1-yapay-zek%C3%A2-490469a10c7b>
- [14] <https://ayyucekizrak.medium.com/veri-odakl%C4%B1-yapay-zek%C3%A2-490469a10c7b>
- [15] https://github.com/mikel-brostrom/Yolov5_DeepSort_Pytorch

